

Лабораторная работа №6
по дисциплине «Управление качеством программного обеспечения»
«Метрики Чидамбера и Кемерера»
Вариант 2
Трусов Михаил, ПИН-42

Задача 2. Определить понятие «Книга». Состояние объекта определяется следующими полями:

- **единый регистрационный номер (длинное целое число);**
- **наименование (строка до 50 символов);**
- **автор (строка до 20 символов);**
- **год издания (целое число);**
- **количество экземпляров (целое число).**

Наименование книги может иметь несколько слов, разделенных пробелами. Увеличить количество экземпляров книги с заданным единым регистрационным номером на 100 единиц.

```
import java.util.ArrayList;  
import java.util.List;  
import java.util.Objects;
```

```
class Book {  
    private long registrationNumber;  
    private String title;  
    private String author;  
    private int publicationYear;  
    private int copiesCount;  
  
    public Book(long registrationNumber, String title, String author, int publicationYear, int  
copiesCount) {  
        this.registrationNumber = registrationNumber;  
        this.title = validateTitle(title);  
        this.author = validateAuthor(author);  
        this.publicationYear = publicationYear;  
        this.copiesCount = copiesCount;  
    }  
  
    private String validateTitle(String title) {  
        if (title == null || title.trim().isEmpty()) {  
            throw new IllegalArgumentException("Название не может быть пустым");  
        }  
        if (title.length() > 50) {  
            throw new IllegalArgumentException("Название не может превышать 50 символов");  
        }  
        return title.trim();  
    }  
}
```

```

private String validateAuthor(String author) {
    if (author == null || author.trim().isEmpty()) {
        throw new IllegalArgumentException("Автор не может быть пустым");
    }
    if (author.length() > 20) {
        throw new IllegalArgumentException("Имя автора не может превышать 20 символов");
    }
    return author.trim();
}

public long getRegistrationNumber() {
    return registrationNumber;
}

public String getTitle() {
    return title;
}

public void setTitle(String title) {
    this.title = validateTitle(title);
}

public String getAuthor() {
    return author;
}

public void setAuthor(String author) {
    this.author = validateAuthor(author);
}

public int getPublicationYear() {
    return publicationYear;
}

public void setPublicationYear(int publicationYear) {
    this.publicationYear = publicationYear;
}

public int getCopiesCount() {
    return copiesCount;
}

public void setCopiesCount(int copiesCount) {
    this.copiesCount = copiesCount;
}

public void increaseCopies(int amount) {
    if (amount < 0) {
        throw new IllegalArgumentException("Количество не может быть отрицательным");
    }
    this.copiesCount += amount;
}

```

```

@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    Book book = (Book) o;
    return registrationNumber == book.registrationNumber;
}

```

```

@Override
public int hashCode() {
    return Objects.hash(registrationNumber);
}

```

```

@Override
public String toString() {
    return String.format("Книга[№%d: '%s' автор: %s, год: %d, экз.: %d]",
        registrationNumber, title, author, publicationYear, copiesCount);
}
}

```

```

class BookManager {
    private List<Book> books;

    public BookManager() {
        this.books = new ArrayList<>();
    }

    public void addBook(Book book) {
        if (book == null) {
            throw new IllegalArgumentException("Книга не может быть null");
        }
        books.add(book);
    }

    public Book findBookByRegistrationNumber(long registrationNumber) {
        return books.stream()
            .filter(book -> book.getRegistrationNumber() == registrationNumber)
            .findFirst()
            .orElse(null);
    }

    public boolean increaseCopiesBy100(long registrationNumber) {
        Book book = findBookByRegistrationNumber(registrationNumber);
        if (book != null) {
            book.increaseCopies(100);
            return true;
        }
        return false;
    }

    public List<Book> getAllBooks() {

```

```

    return new ArrayList<>(books);
}

public int getBooksCount() {
    return books.size();
}
}

public class Main {
    public static void main(String[] args) {
        BookManager manager = new BookManager();
        Book book1 = new Book(1001L, "Война и мир", "Лев Толстой", 1869, 50);
        Book book2 = new Book(1002L, "Преступление и наказание", "Федор Достоевский",
1866, 30);
        Book book3 = new Book(1003L, "Мастер и Маргарита", "Михаил Булгаков", 1967, 25);
        manager.addBook(book1);
        manager.addBook(book2);
        manager.addBook(book3);
        System.out.println("Исходное состояние книг:");
        manager.getAllBooks().forEach(System.out::println);
        long targetRegistrationNumber = 1002L;
        boolean success = manager.increaseCopiesBy100(targetRegistrationNumber);
        if (success) {
            System.out.println("\nКоличество экземпляров книги с номером " +
                targetRegistrationNumber + " увеличено на 100");
        } else {
            System.out.println("\nКнига с номером " + targetRegistrationNumber + " не найдена");
        }
        System.out.println("\nКонечное состояние книг:");
        manager.getAllBooks().forEach(System.out::println);
    }
}

```

Метрики для класса Book

WMC (Weighted Methods per Class):

WMC(Book) = 40 (строка)

NM (Number of Methods):

NM(Book) = 15

DIT (Depth of Inheritance Tree):

DIT = 1 (наследник от Object)

NOC (Number of Children):

NOC = 0 (потомков нет)

CBO (Coupling Between Objects):

CBO = 2 (String, Objects)

RFC (Response For Class):

Методы класса: 15; вызываемые внешние методы: String.trim(), String.length(), Objects.hash()

RFC = 18

LCOM (Lack of Cohesion in Methods):

Атрибуты класса Book (5 полей):

- registrationNumber
- title
- author
- publicationYear
- copiesCount

Анализ использования атрибутов:

Методы, использующие registrationNumber: getRegistrationNumber(), equals(), hashCode(), toString(), конструктор

Методы, использующие title: getTitle(), setTitle(), validateTitle(), toString(), конструктор

Методы, использующие author: getAuthor(), setAuthor(), validateAuthor(), toString(), конструктор

Методы, использующие publicationYear: getPublicationYear(), setPublicationYear(), toString(), конструктор

Методы, использующие copiesCount: getCopiesCount(), setCopiesCount(), increaseCopies(), toString(), конструктор

Практически все пары методов имеют хотя бы один общий атрибут через конструктор или toString. LCOM = 0.

Вывод:

Положительные аспекты:

1. Низкая сложность (WMC = 40) - класс имеет умеренную сложность, что облегчает его понимание и тестирование
2. Высокая связность (LCOM = 0) - все методы класса тесно связаны с его атрибутами, что свидетельствует о хорошей организации ответственностей
3. Простая иерархия наследования (DIT = 1) - класс наследует только от Object, что минимизирует сложность наследования
4. Отсутствие потомков (NOC = 0) - класс не предназначен для расширения через наследование, что упрощает его модификацию

Области для потенциального улучшения:

1. Умеренная связность с другими классами (CBO = 2) - класс зависит от String и Objects, что является приемлемым уровнем связанности
2. Достаточно высокое количество методов (NM = 15) - возможно, некоторые методы валидации можно вынести в отдельный utility-класс